Mock Midterm 3 For CSC182

Object-Oriented Programming in C++ — Practice Exam

Name: ..... ID: .....

Section: ..... Time: 90 minutes

Question 1: The code contains 10 errors, find and correct them:

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount
private:
string owner;
double balance;
public:
BankAccount(string o, double b) {
owner = o;
balance = b;
}
void deposit(double amount) {
balance + amount;
}
void withdraw(double amount) {
if (amount =< balance)
balance -= amount
}
void getBalance() {
return balance;
}
void setOwner(string owner) {
owner = owner;
}
void print() {
cout << owner << ": " << balance << endl;
}
};

int main() {
BankAccount a;
BankAccount b("Ali", 500);
b.deposit();
b.withdraw(100);
b.setOwner("Omar");
cout << b.balance << endl;
b->print();
return 0;
}
```

Question 2: Class Product

1- Create a class named Product with private data members:

- **name of type string**
- **price of type double**
- **quantity of type int**

2- Implement a parameterized constructor for the class that includes default values:

3- Implement the following public member functions:

- **Setter methods — assign product data**
  (price and quantity must not be negative: if the value is negative, do not change the data and print *"Invalid price!"* or *"Invalid quantity!"*)
- **Getter methods — return product data**
- **Method that returns the total value (price × quantity)**
- **Print method — prints all product information**

Question 3: Class Order

1- Create a class named Order with private data members:

- **orderId of type int**
- **item1 of type Product**
- **item2 of type Product**

2- Implement a parameterized constructor for Order:

3- Implement the following public member functions:

- Print method — prints all Order information
- Method to return the cheaper product (the one with the lower price):
- Method that returns the total cost of the order
  (the total value of item1 + the total value of item2):
- Method that takes a product name and returns **true** if the order contains a product with that name:

Question 4: main() Function

1. **Create two Product objects.**

2. **Use the setters:**
   - change the price of the first product to 120
   - try to set the quantity of the second product to −5 (it must be rejected)

3. **Create an Order object.**

4. **Print the information of the Order.**

5. **Show the cheaper product.**

6. **Print the total cost of the order.**

7. **Check if the order contains a product named "Mouse".**

— Good Luck —

Answers

Question 1: The 10 errors and their corrections

#	Line	Wrong	Why	Correct
1	5	`class BankAccount`	The class body must start with `{`	`class BankAccount {`
2	15	`balance + amount;`	Adds but does not store the result	`balance += amount;`
3	18	`amount =< balance`	The operator is written `<=`	`amount <= balance`
4	19	`balance -= amount`	Missing semicolon	`balance -= amount;`
5	21	`void getBalance()`	A `void` function cannot return a value	`double getBalance()`
6	25	`owner = owner;`	The parameter hides the member → assigns the parameter to itself	`this->owner = owner;`
7	33	`BankAccount a;`	There is no default constructor	`BankAccount a("Sara", 0);`
8	35	`b.deposit();`	`deposit` needs an amount	`b.deposit(200);`
9	38	`b.balance`	`balance` is private → use the getter	`b.getBalance()`
10	39	`b->print();`	`b` is an object, not a pointer	`b.print();`

Line 25 compiles, but the name never changes — another fix is renaming the parameter: `void setOwner(string o) { owner = o; }`. Line 33 can also be fixed by giving the constructor default values.

Corrected code

```cpp
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;
public:
    BankAccount(string o, double b) {
        owner = o;
        balance = b;
    }
    void deposit(double amount) {
        balance += amount;
    }
    void withdraw(double amount) {
        if (amount <= balance)
            balance -= amount;
    }
    double getBalance() {
        return balance;
    }
    void setOwner(string owner) {
        this->owner = owner;
    }
    void print() {
        cout << owner << ": " << balance << endl;
    }
};
```

```cpp
int main() {
    BankAccount a("Sara", 0);
    BankAccount b("Ali", 500);
    b.deposit(200);
    b.withdraw(100);
    b.setOwner("Omar");
    cout << b.getBalance() << endl;
    b.print();
    return 0;
}
```

Output:

```
600
Omar: 600
```

Question 2: Class Product

1- Class Product with private data members:

```cpp
#include <iostream>
#include <string>
using namespace std;

class Product {
private:
    string name;
    double price;
    int quantity;
```

2- Parameterized constructor with default values:

```cpp
public:
    // Parameterized constructor with default values
    Product(string n = "None", double p = 0.0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }
```

3- Setters, getters, total value and print:

```cpp
    // Setters (price and quantity cannot be negative)
    void setName(string n) { name = n; }

    void setPrice(double p) {
        if (p >= 0)
            price = p;
        else
            cout << "Invalid price!" << endl;
    }

    void setQuantity(int q) {
        if (q >= 0)
            quantity = q;
        else
            cout << "Invalid quantity!" << endl;
    }

    // Getters
    string getName()  { return name; }
    double getPrice() { return price; }
    int getQuantity() { return quantity; }

    // Return the total value (price x quantity)
    double totalValue() {
        return price * quantity;
    }

    // Print all product information
    void print() {
        cout << "Name    : " << name << endl;
        cout << "Price   : " << price << endl;
        cout << "Quantity: " << quantity << endl;
    }
};
```

Question 3: Class Order

1- Class Order with private data members:

```
class Order {
private:
    int orderId;
    Product item1;
    Product item2;
```

2- Parameterized constructor:

```
public:
    // Constructor using a member initializer list
    Order(int id, Product p1, Product p2) : orderId(id), item1(p1), item2(p2) {}
```

3- Member functions:

```
    // Print all order information
    void print() {
        cout << "Order #" << orderId << endl;
        cout << "--- Item 1 ---" << endl;
        item1.print();
        cout << "--- Item 2 ---" << endl;
        item2.print();
    }

    // Return the cheaper product (lower price)
    Product cheaperItem() {
        if (item1.getPrice() < item2.getPrice())
            return item1;
        else
            return item2;
    }

    // Return the total cost of the order
    double totalCost() {
        return item1.totalValue() + item2.totalValue();
    }

    // Check if the order contains a product with the given name
    bool contains(string n) {
        if (item1.getName() == n || item2.getName() == n)
            return true;
        else
            return false;
    }
};
```

> `totalCost()` reuses `totalValue()` from Product instead of repeating `getPrice() * getQuantity()`. In `contains()`, `||` means **OR**: true if either item matches.

Question 4: main() Function

```cpp
int main() {
    // 1. Create two Product objects
    Product p1("Keyboard", 150, 2);
    Product p2("Mouse", 80, 3);

    // 2. Use the setters
    p1.setPrice(120);        // accepted
    p2.setQuantity(-5);      // rejected -> prints "Invalid quantity!"

    // 3. Create an Order object
    Order o(1001, p1, p2);

    // 4. Print the order information
    o.print();

    // 5. Show the cheaper product
    cout << "\nCheaper item:" << endl;
    o.cheaperItem().print();

    // 6. Print the total cost of the order
    cout << "\nTotal cost: " << o.totalCost() << endl;

    // 7. Check if the order contains a product named "Mouse"
    if (o.contains("Mouse"))
        cout << "The order contains a Mouse." << endl;
    else
        cout << "The order does not contain a Mouse." << endl;

    return 0;
}
```

> Order matters: the setters are called **before** creating the Order, because the Order stores **copies** of p1 and p2. Calling `p1.setPrice(120)` after creating `o` would not change the price inside the order.

Question 4: Program output

```
Invalid quantity!
Order #1001
--- Item 1 ---
Name    : Keyboard
Price   : 120
Quantity: 2
--- Item 2 ---
Name    : Mouse
Price   : 80
Quantity: 3

Cheaper item:
Name    : Mouse
Price   : 80
Quantity: 3

Total cost: 480
The order contains a Mouse.
```

Total cost = 120 × 2 + 80 × 3 = 240 + 240 = **480**. The quantity of Mouse stays 3 because −5 was rejected.

Full Program (Questions 2 + 3 + 4 together)

Compiled and tested with g++ — copy it into any compiler (e.g. onlinegdb.com) to run it.

```cpp
#include <iostream>
#include <string>
using namespace std;

// ================= Class Product =================
class Product {
private:
    string name;
    double price;
    int quantity;

public:
    // Parameterized constructor with default values
    Product(string n = "None", double p = 0.0, int q = 0) {
        name = n;
        price = p;
        quantity = q;
    }

    // Setters (price and quantity cannot be negative)
    void setName(string n) { name = n; }

    void setPrice(double p) {
        if (p >= 0)
            price = p;
        else
            cout << "Invalid price!" << endl;
    }

    void setQuantity(int q) {
        if (q >= 0)
            quantity = q;
        else
            cout << "Invalid quantity!" << endl;
    }

    // Getters
    string getName()  { return name; }
    double getPrice() { return price; }
    int getQuantity() { return quantity; }

    // Return the total value (price x quantity)
    double totalValue() {
        return price * quantity;
    }

    // Print all product information
    void print() {
        cout << "Name     : " << name << endl;
        cout << "Price    : " << price << endl;
        cout << "Quantity: " << quantity << endl;
    }
};
```

Full Program (continued)

```
55    // ================= Class Order =================
56    class Order {
57    private:
58        int orderId;
59        Product item1;
60        Product item2;
61
62    public:
63        // Parameterized constructor (member initializer list)
64        Order(int id, Product p1, Product p2) : orderId(id), item1(p1), item2(p2) {}
65
66        // Print all order information
67        void print() {
68            cout << "Order #" << orderId << endl;
69            cout << "--- Item 1 ---" << endl;
70            item1.print();
71            cout << "--- Item 2 ---" << endl;
72            item2.print();
73        }
74
75        // Return the cheaper product (lower price)
76        Product cheaperItem() {
77            if (item1.getPrice() < item2.getPrice())
78                return item1;
79            else
80                return item2;
81        }
82
83        // Return the total cost of the order
84        double totalCost() {
85            return item1.totalValue() + item2.totalValue();
86        }
87
88        // Check if the order contains a product with the given name
89        bool contains(string n) {
90            if (item1.getName() == n || item2.getName() == n)
91                return true;
92            else
93                return false;
94        }
95    };
```

Full Program (continued)

```cpp
// ================= main() =================
int main() {
    // 1. Create two Product objects
    Product p1("Keyboard", 150, 2);
    Product p2("Mouse", 80, 3);

    // 2. Use the setters
    p1.setPrice(120);        // accepted
    p2.setQuantity(-5);      // rejected -> prints "Invalid quantity!"

    // 3. Create an Order object
    Order o(1001, p1, p2);

    // 4. Print the order information
    o.print();

    // 5. Show the cheaper product
    cout << "\nCheaper item:" << endl;
    o.cheaperItem().print();

    // 6. Print the total cost of the order
    cout << "\nTotal cost: " << o.totalCost() << endl;

    // 7. Check if the order contains a product named "Mouse"
    if (o.contains("Mouse"))
        cout << "The order contains a Mouse." << endl;
    else
        cout << "The order does not contain a Mouse." << endl;

    return 0;
}
```